

Bachelor Thesis

Nsak as Framework for Scenario Based Network Security

Course of study	Bachelor of Science in Computer Science
Author	Frank Gauss (gausf1) and Lukas von Allmen (vonal3)
Advisor	Wenger Hansjürg

Version 1.0 of May 3, 2026

Abstract

One-paragraph summary of the entire study – typically no more than 250 words in length (and in many cases it is well shorter than that), the Abstract provides an overview of the study.

Contents

Abstract	ii
1. Introduction	1
1.1. Motivation	2
1.2. Predecessor Project: Project 2	2
1.3. Project Goals	2
1.3.1. Research Goals	3
1.3.2. Framework	4
1.3.3. Scenarios	6
1.3.4. Evaluation Goals	8
1.3.5. Out of Scope / Optional Feature	9
2. Related Research	10
2.1. Framework Comparison	10
2.1.1. Atomic Red Team	11
2.1.2. Caldera	12
2.1.3. Metasploit	12
2.1.4. NSAK	12
2.2. AI-Research	13
2.2.1. AI and Language Models	13
2.2.2. Model Context Protocol (MCP)	14
2.2.3. Agentic-AI	14
3. Concepts	17
3.1. Framework Resources	17
3.1.1. Device	17
3.1.2. Environment	17
3.1.3. Drill	18
3.1.4. Scenario	18
3.2. Configuration Management	18
3.2.1. Static Configuration	18
3.2.2. Runtime Configuration	19
3.2.3. Resource Configuration	19
4. Methodology	20

5. Implementation	21
5.1. Configuration Management Implementation	22
5.1.1. NSAK Configuration Management	22
5.1.2. Device Configuration Management	27
5.1.3. Scenario & Drill Configuration Management	35
5.2. Reproducible Test Environment	38
5.2.1. Network Topology	39
5.2.2. DMZ-Server	40
5.2.3. Domain Control and File Server	41
5.2.4. Workstations	42
5.2.5. Printer	42
5.2.6. Introduced Vulnerabilities	42
5.3. AI –Scenarios	43
5.3.1. Artificial Intelligence (AI) Agent	43
5.3.2. LangChain Concepts	43
5.3.3. Human Interaction Hook	50
5.3.4. Logging	51
5.3.5. E-Mail Alerting	52
5.3.6. CLI Kill Switch	54
5.3.7. Scenario: AI Agent	55
5.3.8. Scenario: AI Reconnaissance	55
5.3.9. Scenario: AI Intrusion Detection	55
5.3.10. LangChain Agent Chat UI	55
6. Evaluation	56
7. Discussion	57
8. Conclusion	58
Bibliography	59
List of Figures	61
List of Tables	63
List of Listings	64
Glossary	66
A. Project Management	67
A.1. Stakeholders	67
A.2. Agile Management Process	67
A.2.1. Issue Lifecycle:	68
A.2.2. Definition of Ready (DoR)	68

A.2.3. Definition of Done (DoD)	69
A.3. Project Structure	69
A.3.1. Milestones	69
A.3.2. Epics	73
A.3.3. Issues	73
A.4. Sprints / Iterations	73
A.4.1. Planning	73
A.5. Planning and Estimation	74
A.6. Project Roadmap	74
A.7. Sprint Ceremonies	74
A.7.1. Sprint 1: 19.02 –10.03.2026	75
A.7.2. Sprint 2: 05.03 –18.03.2026	78
A.7.3. Sprint 3: 26.03 –09.04.2026	84
A.7.4. Sprint 4: 10.04 –02.05.2026	88
A.7.5. Sprint 5: 03.05 –15.05.2026	89
A.7.6. Sprint 6: 15.05 –28.05.2026	90
A.7.7. Sprint 7: 29.05 –11.06.2026	91
A.7.8. Risk Assessment	93
A.8. Meeting Notes	98
A.8.1. Checkpoint Meeting 1: 26.02.26	98
A.8.2. Checkpoint Meeting 2: 05.03.26	98
A.8.3. Checkpoint Meeting 3: 19.03.26	100
A.8.4. Checkpoint Meeting 4: 02.04.26	101
A.8.5. Checkpoint Meeting 5: 16.04.26	102
A.8.6. Checkpoint Meeting 6: 30.04.26	102
A.8.7. Checkpoint Meeting 7: 14.05.26	103
A.8.8. Checkpoint Meeting 8: 28.05.26	103
A.8.9. Checkpoint Meeting 9: 11.06.26	103
A.9. Variant Decision: Thesis Goals	104
A.9.1. Variant 1: AI vs Manually Built Scenarios	104
A.9.2. Variant 2: Multiple Blue and Red Team Scenarios (Status Quo)	105
A.9.3. Decision	105
A.10. Variant Decision: AI Integration	106
A.10.1. Variant 1: AI-agent in NSAK	106
A.10.2. Variant 2: NSAK as MCP Server	107
A.10.3. Decision	108
A.11. Workshops	109
A.11.1. Brainstorm Scenario Selection	109
A.11.2. NSAK Scenario: Network Discovery - Red Team	111
A.11.3. NSAK Scenario: TLS Downgrade Poodle - Red Team	112
A.11.4. NSAK Scenario: Honeypot - Blue Team	115
A.11.5. NSAK Scenario: Traffic Capture - Red Team	115
A.11.6. NSAK Scenario: Intrusion Detection - Blue Team	116
A.11.7. NSAK Scenario: AI - Purple Team	117

A.12. Self-Hosted LLM	119
A.12.1. Hardware Setup	119
A.12.2. System Setup	119
A.12.3. Evaluation	122
A.12.4. Self-Hosted Base Setup	124
A.12.5. Nginx & Let's Encrypt - Reverse Proxy with SSL Termination	126
A.12.6. Ollama - LLM Server	127
A.12.7. Open WebUI - Web Chat for LLM Servers	130
A.12.8. Drawio MCP - Diagram Tool	133
A.12.9. Netbox - Network Inventory System	136
A.12.10. Grafana / Loki - Logging System	139
A.13. Writing Quality	144
A.14. Test-Environment-Code	145
A.14.1. container-lab	145
A.14.2. Flaws and Vulnerabilities	151

1. Introduction

According to the World Economic Forum's Global Risks Report 2026, cyber insecurity is listed among the top ten global risks and is projected to rise significantly from position 30 to position 5 in the long term, highlighting the growing adverse threat of AI technologies [1].

Complementing this global perspective, the CrowdStrike Global Threat Report highlights a significant rise in AI-enabled adversaries [2].

The absence of, or understaffing of, security experts, paired with the emerging complexity of systems and business models cite Figueredo2024RCVaR, highlights the increasing need for automated tools and frameworks capable of identifying and mitigating emerging cyber threats.

To address this, the *Network Swiss Army Knife (NSAK)*, a modular, open-source framework, is designed to orchestrate security scenarios consisting of reusable attack drills. The framework aims to support automated security testing and adversary emulation in controlled network environments.

The development of the NSAK framework builds upon previous project work, in which the core architecture, simplified configuration management, and a basic scenario execution model were developed. [3].

This thesis further extends the framework by enhancing configuration management for operating the NSAK device, establishing a reproducible test environment, and investigating how agentic AI can support automated red-team and blue-team scenarios.

The thesis is structured as follows: **Chapter 2** reviews related work and analyzes how selected security frameworks integrate AI into their products. **Chapter 3** introduces the fundamental concepts and outlines the key design decisions that form the basis of this work. **Chapter 4** describes the research methodology, including the approach used for evaluation and data collection. **Chapter 5** presents the implementation of the proposed solution, followed by **Chapter 6**, which evaluates its effectiveness in a virtual and physical test environments. Finally, **Chapter 7** discusses the results and their implications, while **Chapter 8** concludes the thesis and highlights potential directions for future work.

Match the structure overview to the actual structure

sharpen research question and contribution

1.1. Motivation

Hypothesis: AI can enhance red and blue team simulations in a network environment by generating and executing scenarios within the Network Swiss Army Knife (NSAK) framework and assisting in the evaluation of detection results. AI can enhance red and blue team simulations in a network environment by generating and executing scenarios within the NSAK framework and assisting in the evaluation of detection results.

1.2. Predecessor Project: Project 2

This section summarizes the predecessor project that led to the bachelor's thesis. The NSAK is a cybersecurity framework that operates with the following resources and concepts: *Devices*, which are physical or virtual machines capable of running the NSAK. The security operation is executed in a network environment and can involve one or more attacks or defense scenarios. The smallest unit in the framework is a drill, which can be orchestrated in a scenario and chained with other drills.

While the modularity and proof of concept were confirmed, several aspects like enhanced configuration management, Graphical User Interface (GUI) or a Representational State Transfer Application Programming Interface (REST API) are mentioned for future work.

The goals of this thesis directly address these open points and are defined in [Section 1.3](#).

Proof
Reading

1.3. Project Goals

For a goal to be considered **S.M.A.R.T.**, it must fulfill the following criteria:

- ▶ **Specific:** What do we want to achieve?
- ▶ **Measurable:** What are the success criteria?
- ▶ **Achievable:** How do we plan to achieve this goal?
- ▶ **Relevant:** Why is it important to achieve this goal?
- ▶ **Time-bound:** Goals categorized as *must* have to be achieved within the thesis, while *should* or *could* goals are considered optional or nice to have.

1.3.1. Research Goals

Compare Frameworks (must)

To identify the USP (unique selling point) of the NSAK framework, we will analyze and compare existing security emulation frameworks. First, relevant frameworks such as MITRE Caldera and Atomic Red Team will be identified and reviewed. Based on this analysis, the identified concepts will be compared with the architecture of the NSAK framework. The comparison will focus on how scenarios and drills are defined, how configuration and parameter handling are structured, how reusable components are organized, how automation is achieved, and which AI capabilities they provide.

The analysis and comparison will be based on the following properties:

- ▶ Concepts
- ▶ Modularity
- ▶ Configurability
- ▶ Automation
- ▶ AI Capabilities

Success criteria:

- ▶ At least two security emulation frameworks are identified and reviewed
- ▶ A matrix comparing the specified properties of the selected frameworks is created
- ▶ The USP of the NSAK framework is identified

AI Integration Research (must)

The goal of this research is to investigate how AI (artificial intelligence) can support automated security testing and adversary emulation. The research evaluates potential integration points where AI could enhance the scenario execution, automation, decision-making, and analysis capabilities of the NSAK framework.

Success criteria:

- ▶ How is AI currently used in cybersecurity automation?
- ▶ Which tasks in adversary emulation can be supported or automated by AI?

- ▶ How could AI interact with the NSAK scenarios and drills (e.g., Agents, MCP)?
- ▶ What architectural changes would be required to integrate AI into NSAK?

1.3.2. Framework

Configuration Management (must)

The goal is to extend the NSAK framework, so that scenarios and drills can expose configurable parameters to increase the flexibility of scenarios and reusability of drills. Parameters can be provided via command-line arguments or YAML Ain't Markup Language (YAML) files and are resolved by the framework into a unified run configuration used for scenario execution.

Success criteria:

- ▶ Scenarios and drills can expose configurable parameters
- ▶ The available parameters can be listed by the Command Line Interface (CLI) with the `--help` option
- ▶ Parameters can be provided via CLI arguments or YAML files
- ▶ The framework resolves the inputs into a run configuration (internal data structure).

REST API (could)

Adding a REST Application Programming Interface (API) to the NSAK framework was already proposed in the predecessor project, as it greatly enhances interoperability with other tools. The priority of this goal depends heavily on the outcome of the goal 1.3.1 and on whether we want to fulfill the goal 1.3.2, which is prioritized as “could”. If the evaluated AI integration requires a REST API (e.g., MCP), this goal needs to be prioritized as “must”; it will be prioritized as “could”.

Success criteria:

- ▶ A REST API specification with feature parity to the CLI is created.
- ▶ The REST API is implemented according to the specification.
- ▶ The API is documented (OpenAPI) and usable by external clients.

- ▶ The REST API remains optional and does not replace the CLI-based interaction.
- ▶ Authentication and Authorization are explicitly out of scope.

MCP Server (could)

The MCP Protocol allows the AI to interact with applications in a standardized way - and could therefore be necessary for implementing AI-based scenarios. If the outcome of the research task [1.3.1](#) highlights the necessity to specify/expose an MCP-Server to interact seamlessly with the NSAK framework, the implementation is a (must), otherwise a (could).

- ▶ A MCP-Server is specified and implemented
- ▶ Documentation how to connect to the MCP is provided.
- ▶ A form of authentication is provided.

Web GUI (could)

The goal is to design and implement a web-based graphical user interface that allows users to interact with the NSAK framework in a user-friendly way. The Web GUI will enable management of scenarios and drills, triggering scenario execution, and visualization of execution results and system status.

Success criteria:

- ▶ A simple Web GUI that interacts with the REST API has been implemented.
- ▶ The Web GUI displays a list of available scenarios and their states.
- ▶ The scenario's results can be viewed on the Web GUI
- ▶ Scenarios can be configured and executed via the Web GUI
- ▶ Authentication and authorization are explicitly out of scope.

1.3.3. Scenarios

AI Scenarios (must)

The goal is to implement artificial intelligence according to the result of 1.3.1 and to explore the use of AI to support red, blue, and purple team exercises in the NSAK framework. The AI will be evaluated for its potential to assist in executing attacks in a scenario and analyzing detection results. The findings will help assess how AI could enhance and coordinate red and blue team activities across a network via the NSAK device.

Success criteria:

- ▶ The AI is integrated into NSAK as a scenario or as part of the framework, depending on the result of 1.3.1.
- ▶ The AI can execute CLI or Python tools to directly or indirectly interact with the attached network.
- ▶ An operator can interact with or stop the running AI.
- ▶ The result artifacts are stored and can be presented or reused for further scenarios.

Reproducible Test Environment (must)

The goal is to design and implement a reproducible and safe test environment for the NSAK framework that enables consistent execution and evaluation of scenarios and drills. The environment should provide a stable basis for testing and comparison of manual and AI-assisted scenarios.

Success criteria:

- ▶ The test environment can be set up reproducibly using a defined configuration or setup procedure with Infrastructure as code (e.g., Ansible)
- ▶ NSAK scenarios and drills can be executed consistently within the environment.
- ▶ The environment supports both manual and AI-assisted scenarios.
- ▶ Scenario results can be collected and analyzed for evaluation purposes.

Network Discovery – Red Team (must)

The goal is to implement a reference red team scenario for network discovery within the NSAK framework. The scenario should simulate reconnaissance activities to identify hosts, services, and network characteristics, and the results should be comparable to the AI-Agent approach.

Success criteria:

- ▶ A red team network discovery scenario is implemented in the NSAK framework.
- ▶ Discovered network information is collected as artifacts.
- ▶ The scenario can be executed reproducibly within the test environment.
- ▶ The results can be compared with those generated by the AI-assisted approach.

Intrusion Detection – Blue Team (should)

The goal is to define and implement a blue team reference scenario for intrusion detection within the NSAK framework. The scenario should allow observation and evaluation of suspicious network activity to assess detection capabilities in the test environment and compare the results with those of the AI approach.

Success criteria:

- ▶ A blue-team intrusion-detection scenario is implemented in the NSAK framework.
- ▶ Detection events or alerts can be observed and documented during scenario execution.
- ▶ Detection results can be compared with results from AI-assisted scenario execution.

1.3.4. Evaluation Goals

AI Scenario Evaluation (must)

The objective is to determine the effectiveness of the AI-based scenario introduced by the goal 1.3.3 within the network environment defined by the goal 1.3.3. The evaluation will assess whether the AI scenario successfully executed the expected red and blue team activities for the provided prompts and the quality of the results.

Success criteria:

- ▶ A methodical approach for the assessment is evaluated and documented.
- ▶ The results of the AI scenario for red and blue team activity are assessed.
- ▶ There is a conclusion on the effectiveness of the AI scenario for red and blue team activities.

AI vs Engineered Scenarios (must/should)

The goal of this evaluation is to compare the quality of AI-based red (network reconnaissance) (must) and blue (network intrusion detection) (should) team scenarios with that of manually engineered scenarios.

The comparison focuses on criteria such as execution reliability, reproducibility, and coverage. The results will help determine the potential benefits and limitations of AI-supported scenarios and provide insights into how AI could enhance future versions of the NSAK framework.

Success criteria:

- ▶ A methodical approach for the comparison is evaluated and documented.
- ▶ The results of the AI scenario are compared against the respective results of the red team scenario (must)
- ▶ The results of the AI scenario are compared against the respective results of the blue team scenario (should)
- ▶ There is a conclusion on the effectiveness of the AI scenario compared to manually engineered scenarios.

1.3.5. Out of Scope / Optional Feature

At the project management meeting on 05.03.26 (see Table [A.16](#) in Appendix A.7), the initial project scope was discussed. During planning, the team proposed using AI-based scenarios and comparing them with the implemented versions, which required reassessing certain milestones.

Running AI-based scenarios does not require a REST API or GUI. A reproducible test setup and further research are needed to properly evaluate this approach.

2. Related Research

2.1. Framework Comparison

Previous research, such as ‘SOK: A Survey of Open-Source Threat Emulators’, compares multiple types of threat emulation frameworks using both quantitative and qualitative metrics [4]. Among the evaluated frameworks, Atomic Red Team, Mitre Caldera, and Metasploit received high scores in categories as environments and operator, environment configuration and scenario execution.

In the following section, the selected frameworks are briefly introduced, and the unique selling point of the NSAK is highlighted. As a point of reference, the cybersecurity community increasingly relies on the concepts of MITRE ATT&CK, which has established itself as a standard framework in the field since its introduction in 2014 [5, 6].

The MITRE ATT&CK is a globally accessible knowledge base of adversary tactics and techniques based on real-world observations [5]. The knowledge base is structured into TTP (Tactics, Techniques, and Procedures), which are organized into 14 attack behavior groups. The TTP is getting continuously updated and extended, which helps to defend against the capabilities of threat actors.

The evaluation of the proposed frameworks is based on information obtained from official documentation [7–9] and the findings of Zilberman et al. [4].

Characteristic	Metasploit Framework	Atomic Red Team	MITRE Caldera
Primary Purpose	Exploitation and penetration testing framework	Security testing through small atomic ATT&CK tests	Automated adversary emulation platform
Relation to MITRE ATT&CK	Partial mapping via modules	Direct mapping to ATT&CK techniques	Built around ATT&CK adversary emulation
Automation Level	Medium (scripts and modules)	Low (manual execution of tests)	High (automated attack chains)
Main Functionality	Exploit development, payload delivery, post-exploitation	Execute small ATT&CK technique tests	Simulate adversary campaigns across hosts
Architecture	Modular framework with exploits, payloads, and auxiliary modules	Collection of scripts/tests organized by ATT&CK techniques	Client-server platform
Typical Use Case	Penetration testing and vulnerability validation	Detection validation and security control testing	Red team automation and purple team exercises
Difficulty Level	Medium-High	Low-Medium	Medium-High

Table 2.1.: Comparison of Metasploit, Atomic Red Team, and MITRE Caldera ([4, 7–9])

As shown in Table 2.1, the key characteristics of the frameworks differ greatly. **Atomic Red Team** provides libraries of attack scripts that implement individual Tactics, Techniques, and Procedures of MITRE ATT&CK. Each attack can be executed independently or chained together into a larger attack scenario. **Mitre Caldera** is a complex, agent-based adversary simulation tool, whereas **Metasploit** is a framework that provides exploits, payloads and auxiliary modules. Further details of the frameworks are presented in the following sections. 2.1.12.1.22.1.3

Table formatting

2.1.1. Atomic Red Team

The Atomic Red Team repository provides documentation and configuration files for each technique. For every TTP, a YAML configuration file defines the execution parameters, while an enhanced Markdown file contains a human-readable description of the attack. This documentation includes a short description of the technique, execution instructions, and additional contextual information [8].

The framework can be executed directly from the command line and provides

features such as logging, detailed error messages, and automatic prerequisite checks. If a technique cannot be executed, for example due to operating system constraints, the framework reports the issue and can optionally attempt to install the required prerequisites [8].

Zilberman et al. recommends deeper cybersecurity knowledge to use Atomic Red Team framework effectively [4]. The Open Source mentality, expandability and easily usable scripts can be a great asset for an independent framework such as the NSAK.

2.1.2. Caldera

Mitre Caldera is also built on the MITRE ATT&CK knowledge base and functions as a command-and-control center. A central server controls agents running on the target system, executes attacks called abilities, and sends the operation results back to the server. Every Ability is a single attack step built on the Tactics, Techniques, and Procedures. The framework ships with a graphical user interface and is primarily used for red team attack simulation, purple teaming, and detection engineering. The additional features, like a training plugin for educational purposes, include facts that can be configured as dynamic safe values, which can be stored and parsed in later steps [7].

With its provided interface and functionality, Caldera is most suitable as a threat emulator, offering a wide range of built-in features, including lateral movement and Command and Control (CNC) communication [4].

2.1.3. Metasploit

Metasploit is a widely used penetration testing framework for testing or attacking systems and exploiting known vulnerabilities. Metasploit follows a sequence of steps to identify a vulnerability in a target system, select a vulnerability for the attack, select an exploit that uses this vulnerability, and deliver a payload that runs on the target system. The configuration via the CLI requires knowledge of the target network and system and requires partial configuration [9, 10].

According to Zilberman et al., an operator requires extensive cybersecurity knowledge to effectively use the framework. To support the operator, the GUI “Armitage”, which is not further evaluated in this work, can be used [4].

2.1.4. NSAK

The NSAK framework, as an independent universal Network Swiss Army Knife, is not confined and can leverage, if needed, different sets of attack frameworks.

An approach could be to use an AI-Enhanced Network Discovery Scenario, which provides the necessary information to configure a Metasploit attack that leads to an attack operation in Caldera and an atomic red team detection script. Hardware-based isolation and network position provide flexibility and should help reduce the operational costs of red and blue teams by reducing simple configuration overhead [3].

2.2. AI-Research

2.2.1. AI and Language Models

Language Model (LM) describes the technique for advancing language intelligence in machines and its development history can be divided into several stages. The first stage, Statistical Language Model (SLM) emerged around the 1990s, and the basic idea is to assume, that such a model can predict the next word. But the fact that an exponential number of transition probabilities are needed, caused a lack of accuracy and led limited success.

To overcome these limitations, Neural Language Model (NLM) was introduced to learn a distributed representation of words and capture more complex patterns, leading to a major breakthrough. Building on this, the advancement of pre-trained language models Pre-trained Language model (PLM), which are first trained on large datasets and then fine-tuned for specific tasks, represented another significant development.

Building on this development, Large Language Model (LLM)s further increases the scale of PLMs by expanding parameter count and computational power [11].

In large language models, parameters are the trainable numerical values that define how the model processes input and generates output. During training, these parameters are continually adjusted to enable the model to learn patterns and language context [12]. The models most of us use today, such as ChatGPT, Claude and Gemini are pre-trained LLMs used for language inference.

These advances led to MCP, a protocol that allows tools and services to expose an interface specifically for AI. Further, the introduction of AI-agents is enabling a new level of autonomy for achieving concrete goals with LLMs. The combination of these technologies is now resulting in systems, where LLMs empower autonomous agents capable of decision-making to execute complex tasks.

2.2.2. MCP

Anthropic, a US software company focused on AI research, introduced the protocol in 2024. The MCP was introduced as an open source standard that enables AI assistants to connect to external systems, including databases, business tools, and development environments [13]. Dynamic tool discovery and schema negotiation is helping clients and agents to access these functions in a standardized manner and allows autonomous interaction. Additionally, MCP allows information to flow in both directions, so the model can send requests, and tools can send updates or events.

This architectural approach shifts the AI integration from static toward an interoperable ecosystem of AI-enabled services. On the other hand, the flexible design and open source approach of MCP unlocks multiple novel and systematic security risks that are in need of enhanced security and governance [14].

2.2.3. Agentic-AI

Agentic Artificial Intelligence (Agentic-AI) can be understood as an advanced machine intelligence that is capable of perceiving, reasoning, and acting with a certain level of autonomy. Traditional systems and agents are more limited to executing predefined commands, whereas Agentic-AI is not restricted in this way. Due to their ability to adapt in real time, they can continuously refine their knowledge, techniques, and data. The ability to access multiple inputs, from natural language processing to sensor data, allows Artificial Intelligence Agent (AI Agent)s to develop a good understanding of their environment [15].

Multi Agent System (MAS) defines a way in which multiple specialist agent coordinate and cooperate with each other to solve a task, that would not be solvable to one agent alone.

Therefore, the agents need a form of interaction that allows them to act reactively, often through a shared mental model, which helps create a common understanding of the complex tasks to be solved. To implement a MAS efficiently, modern design principles such as encapsulation and modularity are important, which are helping to balance resource usage of the system and LLM [16]

Huang et al. provide evidence that Agentic-AI and MAS are capable of solving complex tasks across various sectors, including security and banking. However, these systems also introduce new potential vulnerabilities, both within AI Agent systems themselves and through their interactions with external environments.

The identified vulnerabilities can be categorized into two types: accidental and deliberate. Accidental vulnerabilities include software bugs, logical errors introduced by agents, hardware malfunctions, and poor data quality. Deliberate

attacks, on the other hand, encompass adversarial attacks targeting LLMs, data poisoning, and model theft.

To mitigate these risks, a comprehensive framework consisting of governance mechanisms, proactive monitoring, regulatory compliance, and rigorous testing is required to ensure the secure and reliable operation of agentic systems [17].

Fazit MCP and Agentic AI

In the cybersecurity podcast Risky Business, it is mentioned that MCP is “dead”. The open-source strategy of Anthropic (2.2.2) has led to the development of multiple MCP implementations that were not designed with a strong focus on security, but rather on rapid adoption of a novel technique.

This provocative claim is based on the observation that, in order to handle the excessive MCP tooling, AI Agents may instead rely on direct access to a root shell, as it provides a more efficient way of interaction [18].

Evaluated Frameworks and MCP

Caldera uses a MCP plugin as a bridge between the model and the REST API, allowing the agent to plan and issue commands in a manner similar to a human operator.

Caldera introduces three main research areas. Firstly, a planner determines the order and selection of abilities and can dynamically construct new adversaries based on user requirements through prompt-based interaction.

Secondly, an adversary factory model is used to generate adversaries tailored to the operator’s specific use cases.

Thirdly, a forward-deployed agent is enhanced with Retrieval Augmented Generation (RAG) for Cyber Threat Intelligence (CTI), leveraging Structured Threat Information Expression (STIX)-based data to create reproducible and testable adversaries [19].

The MCP-Server, provided by **Metasploit**, enables a LLM to interact with the Metasploit Framework dynamically. The Interface covers module information (a Metasploit module is a known exploit), exploitation workflows, payload generation, session management and handler management to help manage an attack. The documentation includes a security warning that emphasizes the importance of reviewing every prompt, using a test environment, and being aware of post-exploitation commands. **Metasploit** is a powerful exploitation framework, and the simplicity that AI could bring can also harm. [20]

How and why is this relevant for our thesis? how could we implement it? To which findings should we pay attention? What can we ignore for now?

Elaborate on this, because we will actually not use MCP

I think we are too focused on the term MCP now. We should speak of tool-calling, MCP is just one

The **Atomic Red Team** MCP Server provides an interface that allows AI systems to access and use tests from Atomic Red Team. It enables searching, validating, and optionally executing attack techniques (e.g., based on MITRE ATT&CK). [21]

Outcome Comparison

The framework comparison highlights the difference between the NSAK and the established frameworks. Each compared framework performs a specific task in a complex operation, and the NSAK device can leverage the strengths of multiple aspects of a different set of tools. Building upon these insights, the NSAK is designed to extend and integrate these capabilities into a unified and adaptable system and the change of focus to AI gets further elaborated in this thesis.

What about the other frameworks? What about AI Agents? What can we learn from their approach? How does it affect our strategy?

3. Concepts

3.1. Framework Resources

The NSAK framework is built around four resource types that were originally defined in “Project 2” 1.2 [3]. Each resource type has a distinct role in the framework and is stored as a YAML file in the resource library.

3.1.1. Device

A **Device** resource describes a physical or virtual machine that is capable of running the NSAK framework. It serves as the execution host for scenarios and drills and is typically deployed at a strategic network position to give it the access required for the operations it performs.

A device exposes its network configuration to the framework by declaring its interfaces and their Internet Protocol addresses in its resource YAML. As an alternative, a special device with the id “Unknown” was added, which auto discovers the hosts network configuration. This allows scenarios and drills to resolve interface names and target addresses without hardcoding network details, making the same scenario portable across different devices and network environments.

3.1.2. Environment

An **Environment** represents a specific network topology, including its infrastructure components, hosts, and services [3]. It describes the target network context in which a scenario is intended to operate. For example, a simple client-server setup over a switch, a Wireless LAN access point with connected clients, or a segmented business network.

Environments are referenced by scenarios to make explicit which network context they are designed for, helping operators select and validate the right configuration before executing an operation.

3.1.3. Drill

A **Drill** is a self-contained, reusable unit of execution with a specific, well-defined goal [3]. Its goal can be an active or passive attack step, network discovery, traffic capture, a configuration change, or any other discrete action relevant to a red or blue team activity.

Examples of drills include Address Resolution Protocol spoofing, host discovery via Address Resolution Protocol scan, transparent Transmission Control Protocol proxying, traffic capture with T-Shark, and network configuration tasks such as enabling IP forwarding or Network Address Translation (NAT).

Each drill declares the system and Python packages it requires as well as a typed interface that lists its input arguments and return type. The return value of one drill can be consumed as the input of a subsequent drill within a scenario, enabling data to flow through a chain of execution steps.

3.1.4. Scenario

A **Scenario** describes a concrete red or blue team use case and orchestrates a sequence of drills targeted at one or more environments [3]. Where a drill is a single technical reusable and modular building block, a scenario is an implementation of a specific attack or defense operation.

Like drills, scenarios declare a typed interface with input arguments, allowing operators to parameterise a scenario at execution time via the CLI without modifying the scenario definition itself. This separation keeps scenarios reusable across different network configurations while still allowing runtime flexibility.

3.2. Configuration Management

The NSAK framework distinguishes three configuration layers, each with a different scope and lifetime. Keeping them separate avoids coupling concerns that change at different rates and makes it easier to reason about where a given piece of configuration comes from.

3.2.1. Static Configuration

The static configuration layer involves values which are specific for a deployment and are very unlikely to change after the initial provision of the NSAK device. It mainly establishes the filesystem layout that all other parts of the framework

depend on. For example, where the resource library is located, where runtime data is written, and what the base directory of the installation is.

3.2.2. Runtime Configuration

The runtime configuration layer holds mutable state, which might be changed by an operator via the CLI. It is stored as a YAML file, which is currently loaded at the start of every invocation. If no configuration file exists yet, a default is created automatically on first access, so no explicit initialization step is required from the user.

3.2.3. Resource Configuration

The resource configuration layer is provided by the resource library rather than the framework core. Each resource declares its own configuration inside a YAML file stored alongside its implementation. The framework reads these files to discover and load resources at runtime.

Typed Interface Arguments

Drills and scenarios declare a typed interface in their resource configuration YAML file, that lists the arguments with an explicit type annotation and an optional default value. The framework reads these annotations at runtime and automatically generates the correct CLI options for each resource. This means that adding a new argument to a drill or scenario immediately exposes it as a typed CLI option without requiring any changes to the framework or the CLI code.

Mergeable YAML Structure

All resource YAML files follow a common structure in which resources are nested under a resource-type key (e.g. `drills:`, `scenarios:`). This structure ensures that YAML files from different parts of the library could be merged into a single in-memory data structure without key conflicts, which is a prerequisite for supporting multiple library paths and composite library configurations in the future.

4. Methodology

5. Implementation

5.1. Configuration Management Implementation

The configuration management in NSAK is organised into three distinct layers, each serving a different scope and lifetime.

The **static configuration** layer resolves fixed filesystem paths at import time. It derives all relevant paths from a single base directory, which can be overridden via an environment variable to support different deployment contexts such as Docker containers or automated test environments.

The **runtime configuration** layer holds mutable state that persists across CLI invocations. It is loaded from a YAML file on every command execution and written back whenever state changes, for example, when a different device is loaded. This layer serves as the single source of truth for any information that must be shared between separate CLI calls.

The **resource configuration** layer is defined by the resource library itself. Each resource type –devices, drills, scenarios, and environments– declares its configuration inside a YAML file stored in the library. For devices, this includes network interface assignments, while drills and scenarios additionally declare their typed argument interfaces, which the CLI uses to generate the correct command options at runtime.

5.1.1. NSAK Configuration Management

The NSAK configuration is split into two distinct layers: a static layer that resolves fixed paths at import time, and a mutable runtime layer that is persisted to disk and loaded on every invocation.

Static Configuration

The static configuration is defined in `src/nsak/core/settings.py` and shown in listing 1. It derives all relevant paths from a single `BASE_PATH`, which is either supplied via the `NSAK_BASE_PATH` environment variable or resolved automatically to the repository root at import time. `RUN_PATH` points to the `run/` directory where runtime data such as the persisted config is stored, and `LIBRARY_PATHS` holds the set of directories that are searched for resource libraries. Overriding `NSAK_BASE_PATH` is the primary mechanism to run NSAK from a different base directory, e.g. inside a Docker container or during automated testing.

This section is repeating what's already stated in the concepts chapter...

```
1 import logging
2 import os
3 from pathlib import Path
4
5 from dotenv import load_dotenv
6
7 load_dotenv(os.getenv("NSAK_ENV_FILE", None))
8
9 logger = logging.getLogger()
10
11
12 def get_base_path() -> Path:
13     """
14     Returns the default base path.
15
16     :return:
17     """
18     base_path = os.getenv("NSAK_BASE_PATH", None)
19
20     if base_path is not None:
21         return Path(base_path).absolute()
22
23     return Path(__file__).resolve().parents[3].absolute()
24
25
26 def get_run_path() -> Path:
27     """
28     Returns the run path.
29
30     :return:
31     """
32     run_path = os.getenv("NSAK_RUN_PATH", None)
33
34     if run_path is not None:
35         return Path(run_path).absolute()
36
37     return (get_base_path() / "run").absolute()
38
39
40 def get_library_paths() -> set[Path]:
41     """
42     Returns a list of library paths.
43
44     :return:
45     """
46     library_paths = set()
47
48     default_path = (BASE_PATH / "lib").absolute()
49     if default_path.exists():
50         library_paths.add(default_path)
51
52     library_path = os.getenv("NSAK_LIBRARY_PATH", None)
53     if library_path is not None:
54         library_path = Path(library_path).absolute()
```

Runtime Configuration

The runtime configuration is defined as the “Config” dataclass in `src/nsak/core/configuration`, and holds all state that can change during normal operation, currently the debug flag and the active “Device”. It is persisted as a YAML file at `run/config.yaml` and loaded on every CLI invocation.

To avoid loading the configuration on import, the module-level `config` object is wrapped in a `lazy_object_proxy.Proxy`. The proxy defers the actual `Config.load()` call until the object is first accessed. Since the root CLI group imports `config` on every command invocation (listing 3), this ensures the configuration is always initialised before any command logic runs. If `run/config.yaml` does not yet exist, `Config.init()` is called automatically to create a default configuration and write it to disk, eliminating the need for manual setup on first run.

```
1 from __future__ import annotations
2
3 import dataclasses
4 import logging
5
6 from nsak.core.configuration.ai_configuration import
   ↪ AiConfiguration
7 from nsak.core.configuration.container_info import ContainerInfo
8 from nsak.core.configuration.email_configuration import
   ↪ EmailConfiguration
9 from nsak.core.configuration.loki_configuration import
   ↪ LokiConfiguration
10 from nsak.core.device import Device
11
12 logger = logging.getLogger(__name__)
13
14
15 @dataclasses.dataclass(kw_only=True)
16 class Configuration:
17     """
18     Class for loading and saving the configuration.
19     """
20
21     from ..device.device_manager import DeviceManager
22
23     debug: bool = dataclasses.field(default=True)
24     device: Device = dataclasses.field(
25         default_factory=DeviceManager.get_default,
26     )
27     container: ContainerInfo = dataclasses.field(
28         default_factory=ContainerInfo.load,
29     )
30     email: EmailConfiguration | None = dataclasses.field(
31         default=None,
32     )
33     ai: AiConfiguration | None = dataclasses.field(default=None)
34     loki: LokiConfiguration | None =
   ↪ dataclasses.field(default=None)
35
36     def save(self) -> None:
37         """
38         Shorthand proxy for ConfigManager save.
39         """
40         from .configuration_manager import ConfigurationManager
41
42         ConfigurationManager.save(self)
```

Listing 2: NSAK Runtime Configuration

```
1 import click
2
3 from .ai_agent import ai_agent_group
4 from .config import config_group
5 from .device import device_group
6 from .drill import drill_group
7 from .environment import environment_group
8 from .scenario import kill_all_scenarios, scenario_group
9
10
11 @click.group()
12 def cli() -> None:
13     """
14     CLI root.
15     """
16     # Load the configuration for initialization
17     from nsak.core import config # noqa
18
19
20 cli.add_command(scenario_group)
21 cli.add_command(drill_group)
22 cli.add_command(device_group)
23 cli.add_command(environment_group)
24 cli.add_command(ai_agent_group)
25 cli.add_command(config_group)
26 cli.add_command(kill_all_scenarios, name="killswitch")
```

Listing 3: NSAK CLI Initialisation

Mutations to the runtime configuration always follow the same pattern: a manager method (e.g. `DeviceManager.load()`) updates the relevant field on the `config` object and calls `config.save()` to persist the change. Custom YAML representers are registered for `IPInterface` and `PosixPath` so that these types are serialised as plain strings rather than Python-specific tags. Listing 4 shows an example of the persisted `run/config.yaml` after loading the `bananapi_r4` device.


```
1 debug: true
2 device:
3   id: bananapi_r4
4   name: BananaPI R4
5   path: <base-path>/lib/devices/bananapi_r4
6   author: vonal3@bfh.ch
7   repository: <repository-url>
8   configuration:
9     network:
10      ethernets:
11        lan1@eth0:
12          addresses:
13            10.10.10.30/24:
14              is_target: true
15              is_management: false
```

Listing 4: Persisted Runtime Configuration (run/config.yaml)

5.1.2. Device Configuration Management

In the context of “Project 2”[1.2](#), this resource type was already defined conceptually, and the BananaPI R4 was added as an example in the library under `lib/devices/bananapi_r`. But the resource type was not actually added to the NSAK core, and to manage device configurations, we need to implement the Device resource type first.

Create New Device Resource Type

Similar to the other resource types, a YAML specification for defining devices in the library and multiple classes for representing and managing devices in the core are required. Because the existing resource YAML specifications do not allow merging YAML files, this implementation proposes a new base structure that fulfills this trait. Section [5.1.2](#) describes the necessary changes to the existing YAML specifications. The listing [5](#) shows the initial specification for a device resource, which will get extended later on with a section for configuration management.

```

1 devices:
2   <str:resource-id>:
3     id: <str:device-id>
4     name: <str:device-name>
5     description: <str:device-description>
6     author: <str:author-email>
7     repository: <str:resource-repository>

```

Listing 5: Device Resource YAML Specification

In the table 5.1 below, the required Python classes are listed and described. The classes will be located under `src/nsak/core/device` within their respective files.

Class	File Name	Description
Device	device.py	Dataclass representing a device resource at runtime, closely reflecting the YAML specification.
DeviceLoader	device_loader.py	Loads and returns Device instances from the resource library.
DeviceManager	device_manager.py	Implements business logic and exposes an API for use by the CLI.

Table 5.1.: Device Resource Classes

In “Project 2”[1.2](#), it became already clear that the described classes share a lot of boilerplate code between each other. For this reason, the base classes described in [5.1](#) were added to eliminate duplicated logic, thereby improving compliance with the Don’t Repeat Yourself principles. The classes will be located under `src/nsak/core/resource` within their respective files and the adoption of the existing classes is described in section [5.1.2](#).

Class	File Name	Description
Resource	resource.py	Base class for all resources: Scenario, Drill, Environment and Device.
ResourceLoader	resource_loader.py	Base class which handles searching and loading resources from the filesystem.
ResourceManager	resource_manager.py	Base class which defines common methods for all resource manager classes.

Table 5.2.: Resource Python Classes

To complete the integration of the device resource, the new functionality must be exposed via CLI. The available bash commands are shown in the following listing 6.

```
1 # Show usage, options and subcommands for the device resource
2 nsak device --list
3
4 # List all devices found in the resource library
5 nsak device list
6
7 # Output:
8 › nsak device list
9 ID           NAME           PATH
10 bananapi_r4  BananaPI R4  <repository>/lib/devices/bananapi_r4
```

Listing 6: NSAK CLI: List Devices

Device Configuration Management Implementation

To allow a device to expose its configuration options, the YAML specification needs to be extended accordingly. For now, the focus is only on network configuration, as this is the most important information for the scenarios and drills. The specification was designed so that other sections can be added later. The following excerpt 7 shows the specification for the configuration section of the device resource YAML.

```
1 devices:
2   <str:resource-id>:
3     # ...
4     configuration:
5       network:
6         ethernets:
7           <str:interface-name>:
8             addresses:
9               <str:cidr>:
10                is_target: <boolean>
11                is_management: <boolean>
```

Listing 7: Device Resource Configuration YAML Specification

The following code snippet, located under `src/nsak/core/device/device_loader.py` 8

shows how the YAML configuration gets parsed to the respective datastructures defined in `src/nsak/core/network/configuration.py` and `src/nsak/core/device/device.py`.

```

20 class DeviceLoader(ResourceLoader[Device]):
21     """
22     Finds and loads devices from the library paths.
23     """
24
25     ResourceClass = Device
26
27     @classmethod
28     def _parse_device_configuration(
29         cls, data: dict[str, Any]
30     ) -> DeviceConfiguration | None:
31         if "configuration" not in data:
32             return None
33         network = data["configuration"].get("network") or {}
34         if network == "auto":
35             ethernets = {}
36             for interface in get_network_interfaces():
37                 ethernets[interface.name] = {
38                     "addresses": {
39                         ip: {
40                             "is_target": True,
41                             "is_management": False,
42                         }
43                         for ips in interface.ips.values()
44                         for ip in ips
45                     }
46                 }
47         else:
48             ethernets = network.get("ethernets") or {}
49
50         return DeviceConfiguration(
51             raw=data["configuration"],
52             network=NetworkConfiguration(
53                 ethernets={
54                     interface: EthernetConfiguration(
55                         name=interface,
56                         addresses={
57                             ip: IPConfiguration(
58                                 ip=ip_interface(ip),
59
60                                 ↪ is_target=address.get("is_target",
61                                 ↪ False),
62
63                                 ↪ is_management=address.get("is_management",
64                                31 ↪ False),
65                             )
66                             for ip, address in
67                                 ↪ ethernet.get("addresses",
68                                 ↪ {}).items()
69                         },
70                     ),
71                 }
72             )
73         for interface, ethernet in ethernets.items()
74     }
75
76 )

```

```
12 @dataclasses.dataclass(frozen=True, kw_only=True)
13 class IPConfiguration:
14     """
15     Represents an ip configuration on an interface.
16     """
17
18     ip: IPInterface
19     is_target: bool
20     is_management: bool
21
22
23 @dataclasses.dataclass(frozen=True, kw_only=True)
24 class EthernetConfiguration:
25     """
26     Represents an ethernet node.
27     """
28
29     name: str
30     addresses: dict[str, IPConfiguration]
31
32
33 @dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
34 class DeviceConfiguration:
35     """
36     Represents the device configuration.
37     """
38
39     raw: object
40     network: NetworkConfiguration
41
42
43 @dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
44 class Device(Resource):
45     """
46     Represents a device.
47     """
48
49     NAME = "device"
50     KEY = "devices"
51
52     id: str
53     name: str
54     description: str
55     path: Path
56     author: str
57     repository: str
58     configuration: DeviceConfiguration | None = None
```

Listing 9: Device Configuration Datastructures

The following listing shows the newly available CLI commands for managing device configurations [10](#):

```
1  # List all subcommands for the Device resource
2  nsak device --help
3
4  # List all available devices
5  nsak device list
6
7  # Show the device details and configuration
8  nsak device show <str:device-id>
9
10 # Load a device into in the NSAK configuration
11 nsak device load <str:device-id>
12
13 # Show the currently loaded device stored in the NSAK
14   ↪ configuration
15 nsak device loaded
16
17 # Reset the loaded device
18 nsak device unload
```

Listing 10: Device Configuration CLI Management

Refactor Existing Resources

The implementation of the device resource [5.1.2](#) introduced an improved base structure for the resource YAML specifications and three base classes which are abstracting the common structure and functionality. This section describes the resulting changes in the existing code and resource YAML specifications.

As already stated in section [5.1.2](#) all resource type specific classes were refactored to inherit from their respective base classes as described in table [5.1](#). With the example of the `EnvironmentLoader`, the diff below [5.1](#) illustrates how the abstraction of common properties and methods allows for the removal of a lot of boilerplate code.

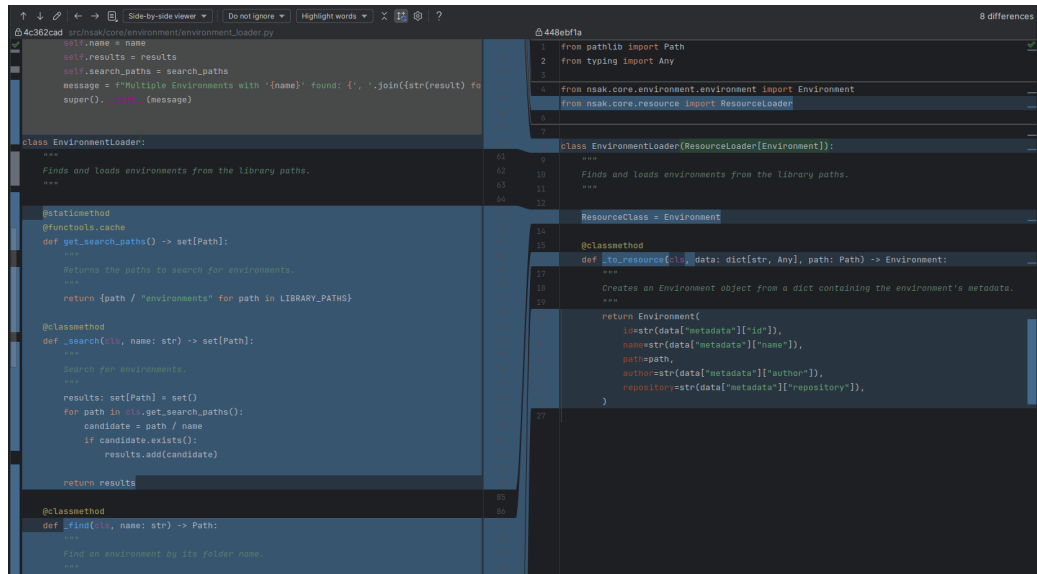


Figure 5.1.: EnvironmentLoader Base Class Refactoring Diff

The structure of all resources was aligned by nesting the resource under their respective resource type key (scenarios, drills, environments, and devices), so that we could merge all YAML resource files into a single library data structure without key conflicts. In the YAML specification [11](#) below, the new base structure derived from the device resource [5.1.2](#) is illustrated. All existing resource YAML files in the library were adjusted accordingly, and thanks to the introduction of the ResourceLoader base class, the required adjustments in the code were minimal.


```
1 # Resource type version was removed, as it was not used anyway
2
3 # scenarios|drills|environments|devices
4 <str:resource-type-key>:
5     # unique resource identifier
6     <str:resource-id>:
7         # The "metadata" key is removed and the children are now
8         ↪ nested directly under the resource
9         name: <str:name>
10        description: <str:description>
11        author: <str:email>
12        repository: <str:link>
13        # ... resource type specific properties, which
14        ↪ previously were at the top level
```

Listing 11: Mergeable Resource YAML Specification

5.1.3. Scenario & Drill Configuration Management

Both the scenario and drill resource types were already fully implemented in “Project 2”[1.2](#). Their YAML specifications already contained an interface section describing argument and return types, but this information was purely documentary and had no effect at runtime.

As part of this project, the `interface.arguments` section was extended [12](#) so that each argument carries an explicit `type` annotation and an optional default value.

```
1 <str:resource-type-key>:
2     <str:resource-id>:
3         # Unchanged properties...
4
5     interface:
6         arguments:
7             <str:argument-name>:
8                 type: <str:type-annotation>
9                 default: <any:default-value>
10        return_type: <str:type-annotation>
```

Listing 12: Updated Interface Specification

The CLI reads these annotations at runtime to generate typed command options

for each drill and scenario automatically, without requiring any additional hand-written code. The `discover_hosts` drill in listing 13 illustrates this: its single interface argument of type `str` is translated directly into a `--interface` option on the generated CLI command.

```
1 drills:
2   discover_hosts:
3     name: Discover Hosts
4     author: vonal3@bfh.ch
5     repository:
6       ↪ https://gitlab.ti.bfh.ch/gausf1-vonal3/swiss-army-knife-network-sniffe
7
8     dependencies:
9       system:
10        - arp-scan
11        - nmap
12       python: [ ]
13
14     interface:
15       arguments:
16         interface:
17           type: str
18         subnet:
19           type: str
20           default: null
21       return_type: NetworkDiscoveryResultMap
```

Listing 13: `discover_hosts` Drill YAML

Scenarios follow the same interface convention and additionally declare the drills and environments they depend on. The `mitm` scenario in listing 14 chains the `discover_hosts`, `arp_spoof`, and `transparent_tcp_proxy` drills together and exposes a single typed interface argument, which the CLI surfaces in the same way as for individual drills.

```
1 scenarios:
2   mitm:
3     id: mitm
4     name: MITM Scenario
5     author: vonal3@bfh.ch
6     repository:
7       ↪ https://gitlab.ti.bfh.ch/gausfl-vonal3/swiss-army-knife-network-sniffer
8
9     drills:
10       - discover_hosts
11       - arp_spoof
12       - transparent_tcp_proxy
13
14     environments:
15       - simple_tcp_client_server
16
17     dependencies:
18       system:
19         - arp-scan
20         - dsniff
21         - iptables
22       python:
23         - scapy
24
25     interface:
26       arguments:
27         interface:
28           type: str
29       return_type: None
```

Listing 14: mitm Scenario YAML

5.2. Reproducible Test Environment

Unless stated otherwise, all referenced implementation artifacts in this section originate from the NSAK repository [3].

The scope of this thesis is confined to the AI based reconnaissance and optionally, the intrusion detection scenarios A.3.1, including the design and implementation of corresponding drills.

To ensure a controlled and safe evaluation of the implementation, a simulated test environment has to be established. This approach prevents unintended interference with production systems and ensures reproducibility during the development and testing phases.

For this exact purpose the “Environment” 3.1.2 resource type was implemented into the NSAK framework. In the predecessor project 1.2, a simplified network topology comprising a client (Alice) and a server (Bob) is used to emulate and analyze a Man In The Mittle (MITM) scenario. This environment is described in a Docker Compose file, located under: `lib/environments/simple_tcp_client_server/compose.yaml`.

The whole setup can be simulated with the following command: `nsak environment simulate simple_tcp_client_server mitm`.

...

While Docker or Podman provides a general-purpose containerization platform, they abstract away most of the lower levels of the network stack and thus lack the support for modeling network topologies. In the existing implementation this was partially addressed, by using the `macvlan` network driver, to enable level 2 networking. In contrast, Containerlab builds upon Docker Compose and optimize its functionalities for networking purposes.

The primary advantages of Containerlab are:

- ▶ It’s ability to create a reproducible containerized network topology based on a simple configuration file
- ▶ Support multiple interfaces nodes which closely resemble real-world network devices (in comparison Docker typically operates with a single network interface).
- ▶ The rapid creation, extension and teardown of complete network environments, ensure that each experiment starts from a clean and controlled state.

In summary, while OCI containers serves as the underlying container engine, Containerlab provides a higher-level network simulation.

The following listing 15 shows an excerpt of a Containerlab topology.yaml file.

Maybe it would make sense to describe this in the project 2 section instead

Basically the nodes are defined and with virtual Ethernet links connected. The bridge represents a multi port switch.

```
1 topology:
2   nodes:
3     node1:
4       kind: linux
5       image: alpine:3.19
6       cap-add:
7         - NET_ADMIN
8       sysctls:
9         net.ipv4.ipforward: 1
10
11     node2:
12       kind: linux
13
14   links:
15     - endpoints: [ "node1:eth1", "bridge:nd1" ]
```

Listing 15: Excerpt of the Containerlab topology definition

5.2.1. Network Topology

The network topology shown in Figure 5.2 illustrates the IP addressing scheme and the interconnections between the individual components.

The `external` node represents an external attacker interacting with the network from the Wide Area Networks (WAN).

The `nsak` container can be deployed in different network segments, enabling the simulation of various attacker positions and reconnaissance scenarios.

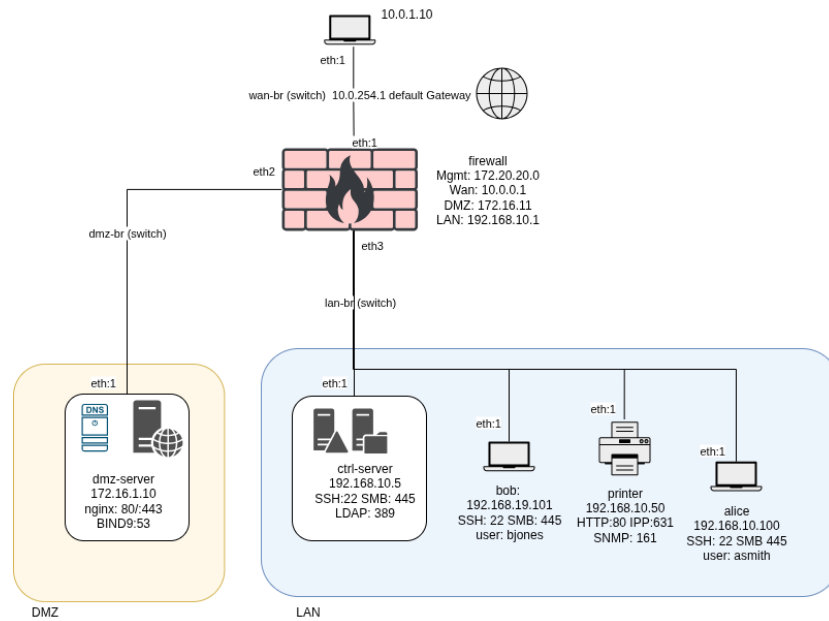


Figure 5.2.: topology-test-environment

5.2.2. DMZ-Server

To evaluate reconnaissance scenarios, common services must run on endpoints.

Berkley Internet Name Domain (BIND) is a complete implementation of the DNS protocol that can be configured through the `named.conf` file as an authoritative name server and resolver.

In the test lab illustrated in 5.2, the functionality of the web server and DNS server was combined for simplicity.

During the De-Militarized Zone (DMZ)-server container build process, a setup shell script is executed.

In the first part, the authoritative BIND `named.conf` file, DNS options, and master zones for forward and reverse lookups are created.

In the forward zone, domain names map to IP addresses, while in the reverse zone, IP addresses map to domain names 48.

add
filepath
and
combine
with
other
sections